

CCP-D1 Final Report — HPC Profiling Results

Course: CS 4110 - High Performance Computing with GPUs

Team Members: Imaad Fazal, Rayyan Imran, & Saleh Mobashar

Github Repository Link: https://github.com/Fastian-afk/HPC_Project

Introduction

This document summarises the profiling results for the KLT Feature Tracker measured on the university HPC platform (NVIDIA RTX 3080). Profiling was performed with gprof on 1920×1080 grayscale inputs, tracking 150 features across 10 frames. The total measured execution time for the run was **1.28 seconds**. The goal of this report is to highlight the top computational hotspots and recommend the primary GPU acceleration targets.

Profiling Summary Table

Function	% Time	Self Time (s)	Calls	Remarks
_convolveSeparate	85.94%	1.10	63	Dominant bottleneck — #1 GPU target (separable convolution)
_KLTSelectGoodFeatures	7.03%	0.09	1	#2 GPU target (Shi-Tomasi corner detection)
_quicksort	3.12%	0.04	1 (+1,147,784 internal calls)	#3 target (sorting for feature prioritization)
_interpolate	0.00%	0.00	2,764,090	Massive parallelism potential — bilinear sampling
_computeIntensityDifference	0.00%	0.00	10,775	Optical-flow residual computation (parallel reduce)
_computeGradientSum	0.00%	0.00	8,715	Gradient accumulation (warp-level reductions)

Note: 'Self Time' is the time attributed to the function itself (not including children). Percentages come from the gprof flat profile; entries with 0.00% may still have very high call counts and strong GPU suitability.

Key Findings

- Total execution time: **1.28 seconds**. The bulk of runtime ($\approx 86\%$) is consumed by separable convolution.
- `_convolveSeparate` (85.94%, 1.10 s) is the highest-priority GPU target; expected large speedup with shared memory tiling and kernel fusion.
- `_KLSelectGoodFeatures` (7.03%, 0.09 s) is the second priority; parallel corner-response computation and top-N reduction recommended.
- `_quicksort` (3.12%, 0.04 s) can benefit from GPU radix/bitonic sort for larger feature counts.
- `_interpolate` (2,764,090 calls) and other high-call support functions indicate substantial SIMD/texture unit opportunities despite negligible CPU time.

Conclusion

The profiling results show that targeted GPU acceleration of a few functions will cover the majority of execution time. Porting `_convolveSeparate` first (with shared memory and possible kernel fusion with gradient computation) will deliver the largest application-level benefit. Follow-up work should implement and validate CUDA kernels for the top targets, measure end-to-end speedup, and iterate with memory and launch-configuration optimizations (streams, pinned host memory, texture units).